



密码学引论实验报告

姓名: 江李阳, 王健一, 邱泽辉

学号: 202400460042, 202400460037, 202400460046

专业: 密码科学与技术

日期: May 14, 2026

Contents

1	实验目的	2
2	实验环境	2
3	实验原理	2
3.1	密文分组链接模式	3
3.2	CBC 模式解密	3
3.3	明文填充	3
3.3.1	PKCS#5 填充标准	3
3.3.2	PKCS#7 填充标准	3
3.4	Padding oracle 攻击	4
3.5	攻击本质	5
4	目标密文与分组	5
4.1	密文分组	5
5	攻击算法设计	6
5.1	整体攻击思路	6
5.2	基础版算法的主线	6
5.3	按块恢复的执行过程	7
5.4	实际提交密文格式	7
5.5	算法复杂度	7
6	代码实现说明	7
6.1	程序结构	7
6.2	主要实现流程	8
6.3	实现体会	8
7	运行结果与破解明文	8
7.1	分组恢复结果	9
7.2	完整明文恢复结果	10
8	实验分析与总结	10
8.1	实验结果分析	10
8.2	实现过程中的体会	10
8.3	安全启示	11
9	附录	12
9.1	实验运行代码	12

1 实验目的

1. 理解 AES-128-CBC 模式的加解密结构，明确明文分组、密文分组和初始向量 IV 之间的关系；
2. 掌握 PKCS#7 填充规则，理解填充合法性判断如何泄露明文相关信息；
3. 编写 Padding Oracle 攻击程序，利用服务器返回的填充状态恢复 CBC 解密过程中的中间状态；

2 实验环境

本实验在本地编写攻击程序，并通过访问实验指导书给出的解密服务器完成 Padding Oracle 攻击。实验过程中，攻击者不需要知道 AES 密钥，只需依据服务器返回的状态判断解密结果的 PKCS#7 填充是否合法。

本小组成员学号末位分别为 2、7、6，因此目标密文编号按下式确定：

$$2 + 7 + 6 = 15 \equiv 5 \pmod{10}$$

故本实验选择 5 号目标密文进行攻击。

Table 1: 实验环境

项目	配置或说明
操作系统	Windows 11
编程语言	Python 3
开发工具	Visual Studio Code
加密算法	AES-128-CBC
填充标准	PKCS#7
目标密文编号	5 号密文
密钥组	Key_2
解密接口	http://10.102.33.67:8208/dec_2?data=

实验指导书中的解密服务器会对输入密文进行 AES-128-CBC 解密，并检查解密后明文是否满足 PKCS#7 填充规则。若填充正确，服务器返回 HTTP 200；若填充错误，则返回 HTTP 500。因此，本实验通过构造查询密文并观察服务器反馈状态，逐步恢复目标密文对应的明文。

实验材料中给出的 5 号密文属于 Key_2 加密所得密文，因此程序访问的解密接口为 dec_2。实验运行时需要保证本机能够访问学校实验服务器，通常需要处于校园网环境或连接学校 VPN。

3 实验原理

本实验的核心原理是 Padding Oracle 攻击。该攻击针对采用 CBC 工作模式并使用 PKCS#7 填充的分组密码系统。当解密服务器会向攻击者反馈“填充是否合法”时，即使攻击者不知道 AES 密钥，也可以通过构造密文并反复查询服务器，逐字节恢复目标密文对应的明文。

3.1 密文分组链接模式

CBC (Cipher Block Chaining, 密文分组链接) 模式的基本思路是: 加密当前明文分组时, 先将它与前一密文分组异或, 再送入分组密码算法。设明文分组为 $m_1, m_2, \dots, m_s$, 密文分组为 $c_1, c_2, \dots, c_s$, 初始向量为 IV , 并记 $c_0 = IV$, 则 CBC 加密关系为

$$c_i = Enc_k(m_i \oplus c_{i-1}), \quad i = 1, 2, \dots, s.$$

由此可以看出, 前一密文分组会影响后一分组的加密结果。例如, m_1 不仅影响 c_1 , 还会通过链式关系继续影响后续分组。因此 CBC 模式具有明显的前后依赖关系, 加密时不能像 ECB 模式那样对所有明文分组完全独立地并行处理。

3.2 CBC 模式解密

由 CBC 模式的加密关系可知, 解密时必须使用 $c_0 = IV$, 因此实际传输和保存的密文通常写作

$$IV \parallel c_1 \parallel c_2 \parallel \dots \parallel c_s.$$

设密文分组 c_i 对应的明文分组为 p_i , 则有

$$p_i = Dec_k(c_i) \oplus c_{i-1}, \quad i = 1, 2, \dots, s,$$

其中 $c_0 = IV$ 。将 $Dec_k(c_i)$ 记作中间状态 A_i , 则上式可以写成

$$p_i = A_i \oplus c_{i-1}. \quad (1)$$

因此, 只要能够恢复出中间状态 A_i , 再与真实前一密文分组 c_{i-1} 异或, 就可以得到当前密文分组对应的真实明文分组 p_i 。

3.3 明文填充

AES 的标准分组长度为 128 bit, 即 16 字节, 但待加密明文 m 的长度不一定正好是 16 字节的整数倍。因此在加密前, 需要先对明文进行填充 (padding), 使填充后的长度成为分组长度的整数倍。

3.3.1 PKCS#5 填充标准

PKCS#5 以 8 个字节为一个分组。若明文长度不足一个分组, 则补足相应数量的字节; 若明文长度刚好等于一个完整分组, 也要额外补充一个新的分组。填充值等于填充字节数。例如, 若需要填充 7 个字节, 则补入 7 个 0x07; 若刚好是一个分组, 则补入 8 个 0x08。

3.3.2 PKCS#7 填充标准

PKCS#7 的规则与 PKCS#5 类似, 但分组长度不再限制为 8 字节, 而是适用于 1 ~ 255 字节的分组。对于 AES-128 而言, 一个分组为 16 字节。若某个明文分组还差 3 个字节补满, 则填充后的结果形如

$$M_{1 \sim 13}, 0x03, 0x03, 0x03.$$

因此, 解密服务器在解密完成后, 会检查最后若干字节是否满足 PKCS#7 填充规则。Padding Oracle 攻击正是利用了这个检查结果是否可区分这一点。

3.4 Padding oracle 攻击

Padding Oracle 攻击是一种选择密文攻击。设最后一个目标密文分组为 c_s ，攻击者先取一个 16 字节分组¹

$$r = r_0 r_1 \cdots r_{15}$$

作为伪造的前一分组，并将它与 c_s 一起提交给解密服务器。实验指导书中的基本形式是 $r \parallel c_s$ ；而在本实验环境中，为了满足服务器接口格式，程序实际会保留必要前缀，使目标块仍然处在整段提交密文的最后一个分组位置。无论前面前缀如何，服务器最终检查的仍然是

$$Dec_k(c_s) \oplus r = A_s \oplus r$$

这一分组的填充是否满足 PKCS#7 规则。具体的检查方式可以参考 <https://github.com/Legrandin/pycrypto>

```
1 if style == 'pkcs7':
2     if padded_data[-padding_len:] != bchr(padding_len)*padding_len:
3         raise ValueError("PKCS#7 padding is incorrect.")
```

如果填充长度是 `padding_len`，那么服务器会检查明文的最后 `padding_len` 字节是否都是 `padding_len`

如果填充长度是 2，那最后两个字节都应该是 0x02

这次实验服务器给出的反馈是这样的：

- 如果不满足填充规则，则返回填充错误信息（HTTP 500）；
- 如果满足填充规则，则返回正确信息（HTTP 200）。

值得注意的是，服务器不会去检查解密的结果是否正确，也就是说解密的结果无论如何，服务器都不会报错。针对这点我们可以做几个测试：

(1) 原始密文解密：

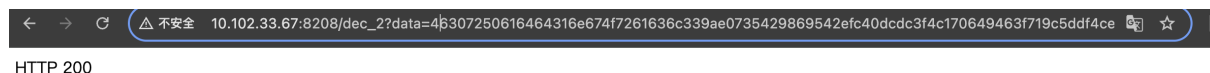


Figure 1: 用原始密文解密

将原始密文解密，状态是 200

(2) 修改但不破坏填充结构: 如果我们修改 IV 的第一个数字，比如把 4 改成 5

¹注：实验指导书这里写的是 $r_1, r_2, \dots$ ，但为了和代码适配，采用从 0 开始的方式



Figure 2: 修改 iv

会发现没有变化。很明显这时解密结果与第一次不同,但服务器并不针对解密结果反馈是否正确,只是对填充规则是否则正确做出反应。

- (3) 修改但破坏填充结构: 如果修改最后一块密文, 比如把最后一个数字从 4 改成 5(这样会破坏填充规则)



Figure 3: 修改最后一块密文

此时 HTTP 500, 因为这样破坏了填充规则这说明服务器只会检测填充, 而不会对我们传入的密文做一些检测, 那么这说明我们是控制密文的, 有点类似选择密文攻击通过控制密文可以逐步反推出中间状态 A_s

3.5 恢复算法

不妨设倒数第二块密文为

$$r = r_0 r_1 \dots r_{15}$$

它对应的明文是

$$p = p_0 p_1 \dots p_{15}$$

其中

$$p_i = a_i \oplus r_i$$

3.5.1 测出填充长度

一个很重要的事情是, 最后一块明文本身是有填充的, 也就是说最后有好几个字节是相等的, 且都等于 p_{15} , 如果 $p_{15} = 0x02$, 那么 $p_{14} = p_{15} = 0x02$

鉴于前面说的, 修改倒数第二块密文并不会影响服务器的反馈, 那么我们完全可以去控制 r 来得到 p 的一些信息

值得注意的是, 实验指导书这一块说要遍历 r_{15} 直到符合填充规则, 但是经过实验发现 r_{15} 是没有必要修改的 (不过在恢复前面的密文的时候则不然), 直接取真实的就行

因为真实的 r_{15} 就是使得 p_{15} 符合填充规则的

我们的思路就是，控制 r 去试探一下到底有多少个字节相等，字节相等的数量其实就是填充长度

这边有两个方向

1. 修改 $r_0, r_1, \dots$

如果修改到 r_j 的时候，不符合填充规则，那么这说明从 r_j 到 r_{15} 都是填充字节

那么 $\text{padding_length} = 16 - j$

2. 修改 $r_{14}, r_{15}, \dots$

这样一定不符合填充规则，但是如果修改到 r_j 的时候符合填充规则了，那说明从 r_{j+1} 到 r_{15} 都是填充字节

那么 $\text{padding_length} = 15 - j$

代码里实现用的是第二个，因为填充长度一般来说比较小，反过来会快一些，而且从代码角度方向 2 比较自然

只需要设置 padding_length 的初始值为 1，然后如果不符合规则就加 1 就行了

3.5.2 恢复部分中间状态

利用原始填充恢复部分中间状态：假设根据上一步判断出填充长度为 t ，则末尾 t 个字节满足

$$a_i \oplus r_i = t, i = 16 - t, \dots, 15$$

3.5.3 恢复前一字节

那前面的 a 怎么恢复呢，既然我们已经恢复了后面的 a ，那我们完全可以修改 r 让前面的 a 也变成填充字节，这样我们就可以通过异或求出来了

只需要让填充字节加 1 即可

假设当前已经恢复出 $a_j, a_{j+1}, \dots, a_{15}$,

$$r'_i = a_i \oplus (t + 1), i = j, \dots, 15$$

穷搜 r'_{j-1} 的取值，直到服务器返回填充正确。

此时

$$a_{j-1} = r'_{15} \oplus (t + 1)$$

用同样的方法继续向前推进，就可以依次恢复前面的 a 了，直到恢复所有 a

3.5.4 恢复最后一块明文

当最后一个目标分组 c_s 的全部中间状态都恢复完成后，便得到

$$A_s = a_0 a_1 \cdots a_{15}.$$

再结合真实前一密文分组 c_{s-1} ，利用公式

$$p_s = A_s \oplus c_{s-1}$$

即可恢复出最后一个分组的明文

3.5.5 如何恢复前面的明文

实验指导书只给了恢复最后一块的方法，恢复前面的话，略有不同，不能一概而论

我们只需要把要恢复的明文放到最后就行了

前面的明文没有填充，这是区别于最后一块的关键特征，所以 r 不能用真实的

所以我们没有必要去测填充长度，我们必须穷搜 r_{15} 使得它异或 a_{15} 为 $0x01$ ，不然就无法通过填充规则

(但这里有例外，万一明文的最后刚好就是 $0x02$ ，那如果 $r_{15} \oplus a_{15} = 0x02$ ，那也是符合填充规则的)

一开始写代码的时候我认为恢复前面的明文得用另一套代码，但是仔细考虑上面的例外之后，发现是可以和最后一块明文的方法统一的

我们穷搜 r_{15} ，一定存在一个值使得它通过填充规则，那么此时我们就可以认为这个明文是有填充的

也就是说虽然原本这个明文没有填充，但是我们可以修改 r_{15} 让它变得有填充，从而递归转化成近似最后一块明文的问题

大概率这个填充是 $0x01$ ，但是考虑上面的例外的话，也有很小的概率是 $0x02, 0x03$ ，所以我们如果把例外纳入考量，那就也可以去测填充长度

这样就统一了

3.6 攻击本质

Padding Oracle 攻击并不是直接破解 AES 算法本身，也不是恢复 AES 密钥，而是利用了解密系统实现中的信息泄露。具体来说，服务器把“填充正确”和“填充错误”以不同的状态返回给攻击者，使攻击者能够通过大量选择密文查询，逐步推断 CBC 解密中间状态，最终恢复明文。

因此，本实验说明：只提供机密性而缺少完整性保护的 CBC 加密实现是不安全的。在实际系统中，不能将可区分的 padding 错误直接暴露给外部用户，更安全的做法是使用认证加密模式，或者在解密前先进行消息认证。

4 目标密文与分组

本实验选择的目标密文编号为 5，对应的解密接口为 `dec_2`。目标密文采用 AES-128-CBC 模式加密，并使用 PKCS#7 填充。由于 AES 的分组长度为 16 字节，因此需要按照每 16 字节，即每 32 个十六进制字符，对目标密文进行分组。

4.1 密文分组

目标密文的整体结构为：

$$IV \parallel C_1 \parallel C_2 \parallel C_3$$

其中， IV 为初始向量， C_1, C_2, C_3 为三个密文分组。按照 16 字节分组后得到：

Table 2: 5 号目标密文分组结果

分组	十六进制表示
IV	46307250616464316e674f7261636c33
C_1	9ae0735429869542efc40dcdc3f4c170
C_2	649463f719c5ddf4ce8c6d1ef0e5d41a
C_3	c5d137629e3fe1340cfaad7e21d65d14

在 CBC 解密过程中, IV 可视为第 0 个密文分组, 即 $C_0 = IV$ 。因此三个明文分组的恢复关系分别为:

$$P_1 = Dec_k(C_1) \oplus IV, \quad (2)$$

$$P_2 = Dec_k(C_2) \oplus C_1, \quad (3)$$

$$P_3 = Dec_k(C_3) \oplus C_2. \quad (4)$$

后续攻击过程的目标就是分别恢复 $D_k(C_1)$ 、 $D_k(C_2)$ 和 $D_k(C_3)$ 的中间状态, 再与对应的前一分组异或, 得到当前密文分组对应的明文, 最终拼接出完整明文。由于 Padding Oracle 攻击总是针对“提交密文中的最后一个分组”进行判断, 因此在实际查询时, 需要构造密文使当前待恢复的目标块位于查询密文的末尾位置。

5 代码实现说明

本实验的攻击算法按照实验指导书中的思路设计, 并与 `attack\_slow.py` 对应。第 3 节已经说明了 Padding Oracle 攻击的原理, 本节整理了算法执行流程

5.1 整体攻击思路

程序首先将目标密文拆分为

$$IV, C_1, C_2, C_3$$

四个分组, 然后从最后一个目标分组开始, 逐块恢复对应的中间状态和明文。

5.2 算法流程

1. 取该目标分组真实前一密文分组作为初始参考分组 (如果目标分组不是最后一块, 需要穷搜前一块的最后一个字节);
2. 通过逐个修改参考分组的字节, 判断原始明文末尾的填充长度;
3. 根据已知填充长度, 先恢复中间状态末尾字节;
4. 修改前一块的字节, 构造新的填充值;
5. 穷搜恢复前一个中间状态字节;
6. 当整个中间状态恢复完成后, 再与真实前一密文分组异或, 得到目标明文分组。

这一设计与实验指导书中的推导是一致的, 也和第 3 节中的原理说明完全对应。

5.3 按块恢复的执行过程

对每个目标分组，程序都会先获得该分组对应的中间状态后缀，再逐步向前扩展。这个过程与原理部分的例子一致：先根据服务器反馈判断原始填充长度，再利用已知后缀去构造新的合法填充，逐字节恢复更多中间状态。等到整个中间状态都恢复完成后，再与真实前一密文分组异或，即可得到当前明文分组。

5.4 实际提交密文格式

理论上，Padding Oracle 攻击只用到两个分组。但在本实验环境中，服务器必须要求全部提交，但是只需要把前面都设为 0 就行了，对攻击没有影响

5.5 算法复杂度

对每个待恢复字节而言，最坏情况下需要枚举 256 种可能取值。每个 AES 分组有 16 个字节，因此恢复一个分组最多需要

$$16 \times 256 = 4096$$

次 oracle 查询。

本实验目标密文除 IV 外共有 3 个数据分组，因此理论上的查询上界为

$$3 \times 16 \times 256 = 12288.$$

实际运行时，并不是每个字节都会枚举满 256 次，因此真实查询次数通常小于该上界。与此同时，程序是按块逐步恢复的，因此一旦某个分组恢复完成，就可以立即得到对应的明文结果，便于随时检查程序是否运行正常。

6 运行结果与破解明文

本节给出程序运行结果以及最终恢复得到的明文。实验开始时，先将目标密文拆分为 IV, C_1, C_2, C_3 四个分组，并验证原始密文能够通过服务器的 PKCS#7 填充检查。服务器返回填充正确，说明 oracle 接口可正常使用，后续攻击可以继续进行。

运行结果如下图：

```

third_experiment > report > code > attack_slow.py > ...
1 import requests
2 from tqdm import *
3 ciphertext = '46307250616464316e674f7261636c339ae0735429869542efc40dc3f4c170649463f719c5ddf...'
4 cipher = [ciphertext[:32], ciphertext[32:64], ciphertext[64:96], ciphertext[96:128]]
5 #获取第i到j字节
6 def get_bytes(s,i,j):
7     if j < i:
8         return ""
9     else:
10        return s[2*i:2*i+2*(j-i+1)]
11 #str转成列表
12 def get_list(s):
13     lst = [s[i:i+2] for i in range(0, len(s), 2)]
14     return lst
15 #列表转成str
16 def list_to_str(lst):
17     return ''.join(f'{x & 0xff:02x}' for x in lst)
18 p = []
19 for idx in range(2, -1, -1):
20     ...

```

infinite@infiniteMacBook-Air ~/Documents/crypto_introduction main ± /opt/homebrew/bin/pyt...
hon3 /Users/infinite/Documents/crypto_introduction/third_experiment/report/code/attack_slow.py
填充长度是 1
0% | 0% | 0% | 32% | 8% | 18% | 7% | 4% | 5% | 36% | 35% | 9% | 23% | 11% | 32% | 19% | 100%
flag{555msdb6ax9bh0eq1vh1gfcx2cyr1ws666}
infinite@infiniteMacBook-Air ~/Documents/crypto_introduction main ±
行 34, 列 21 空格: 4 UTF-8 LF Python 已达到内联建议配额 Python 3.14.4 (homebrew)

Figure 4: 代码运行结果

6.1 分组恢复结果

程序依次对三个目标密文分组进行恢复，最终得到的明文分组结果如表 3 所示。

Table 3: 各明文分组恢复结果

分组	明文十六进制表示	ASCII 表示
P_1	666c61677b353535 6d73646236617864	flag{555msdb6axd
P_2	3962683065716c76 683167666378326b	9bh0eq1vh1gfcx2k
P_3	6379723177733636 367d060606060606	cyr1ws666}

从表中可以看到，第三个明文分组的末尾为 06 06 06 06 06 06。这说明原始明文在最后补入了 6 个 PKCS#7 填充字节，因此这些字节不属于真实消息内容。

6.2 完整明文恢复结果

将三个明文分组顺次拼接后，可以得到带填充的完整明文，其十六进制表示为

```
666c61677b3535356d73646236617864
3962683065716c76683167666378326b
6379723177733636367d060606060606
```

去除末尾 6 个字节的 PKCS#7 填充后，真实明文的十六进制表示为

```
666c61677b3535356d73646236617864
3962683065716c76683167666378326b
6379723177733636367d
```

将其按 UTF-8 字符串解码，最终恢复得到 5 号目标密文对应的明文为

```
flag{555msdb6axd9bh0eqlvh1gfcx2kcyr1ws666}
```

这一结果与三个分组的逐块恢复结果是能够相互对应的，也说明程序恢复出的中间状态和最终明文是一致的。

7 实验分析与总结

从实验结果可以看出，Padding Oracle 攻击并不是去破解 AES 算法本身，而是利用了解密系统在实现上的信息泄露。攻击者虽然不知道密钥，但只要能够反复提交构造密文，并区分服务器返回的是“填充正确”还是“填充错误”，就可以逐步恢复 CBC 解密过程中的中间状态，最后还原出真实明文。

7.1 实验结果分析

本实验能够成功，依赖于以下几个条件同时成立：

- 目标系统使用了 CBC 分组模式，并采用 PKCS#7 填充；
- 解密服务器在解密后会检查填充是否合法；
- 攻击者能够区分服务器返回的不同错误状态。

这三个条件一旦同时满足，服务器实际上就成了一个“padding oracle”。虽然它没有直接把明文或密钥返回出来，但每一次反馈都在泄露一部分关于解密结果的信息。攻击者把这些信息不断累积起来，就能一点一点地恢复出完整明文。

7.2 实现过程中的体会

这次实验中比较明显的一点是：原理本身并不算太难，真正容易出错的是实现细节。例如，填充长度的判断、字节下标的对应关系、中间状态与真实明文的区分，这些地方只要错一个，后面的恢复过程就会全部受到影响。

7.3 安全启示

本实验说明：只有机密性、没有完整性保护的 CBC 加密实现是不安全的。为了防御 Padding Oracle 攻击，实际系统中通常要避免把可区分的 padding 错误直接暴露给外部用户，更安全的做法包括：

- 使用带认证的加密模式，如 GCM、CCM 等；
- 在解密前先做消息认证，而不是先解密再判断 padding；
- 对外统一错误返回，避免泄露“是 padding 错误还是其他错误”这类差异信息。

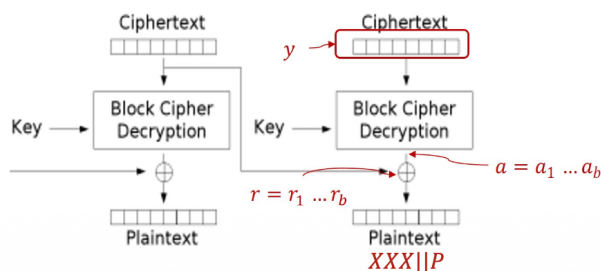
总的来说，这次实验不仅加深了对 CBC 模式和 PKCS#7 填充规则的理解，也更直观地说明了：密码算法本身安全，并不等于系统实现就一定安全。

8 实验指导书可能存在的问题

8.1 字与字节

解决方案——基本思想

- Padding oracle攻击是一种选择密文攻击。假设所截获密文的最后一个分组为 y ，随机选择一个分组 $r = r_1 r_2 \dots r_b$ 作为密文的倒数第二个分组，其中 r_i 为一个字(word)，将 $r||y$ 作为密文发送给解密服务器。
- 解密服务器运行CBC解密程序并判断填充是否合法，例如，判断是否满足PKCS#7规则。
 - 如果不满足填充规则，则返回填充错误信息，如HTTP 500 server error
 - 如果满足则填充规则，则返回正确信息，如HTTP 200
- 根据解密服务器反馈信息，进一步确定CBC解密中间状态值 a



令 $Dec(y) = a = a_1 \dots a_b$ ，则“正确”的CBC解密的最后一个分组应满足如下形式：

$Dec(y) \oplus r = a_1 \dots a_b \oplus r_1 \dots r_b = XXX||P$
其中 P 为正确的填充值，即 P 满足形式 $0x01$ 或 $0x02||0x02$ 或者 $0x03||0x03||0x03$ 或者...

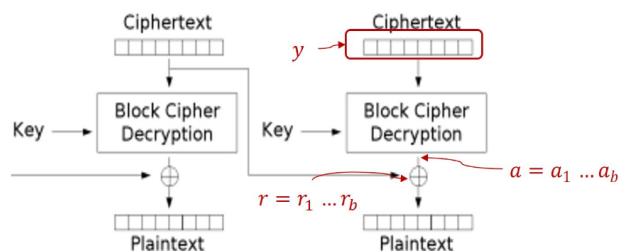
Figure 5: 错误 1

此处 r_i 是一个字节 (Byte), 不是一个字 (word)

8.2 没有必要的步骤

(1) 如何找到满足上式的 r ?

$$Dec(y) \oplus r = a_1 \dots a_b \oplus r_1 \dots r_b = XXX||P$$



- 思想：从最后一个分组入手，穷举猜测 r 使其满足CBC最后一个或多个分组的输出填充满足填充规则，例如，最后一个字节满足 $a_b \oplus r_b = 0x01$ 。（注意， y 的真实填充可能并非 $0x01$ 。）
- 具体方法：穷举搜索 r_b ，并将对应的 $r||y$ 发送给解密服务器，直到CBC解密反馈填充正确信息，此时对应的填充可能是 $0x01$ 或 $0x02||0x02$ 或者 $0x03||0x03||0x03$ 等情况，
- 注意， y 的真实明文填充可能并非此时对应的填充。（但该情况下很有可能找到的是只有1个字节填充，即 $a_b \oplus r_b = 0x01$ 。）

Figure 6: 错误 2

正如前面实验原理所说，在恢复最后一块的时候，这个步骤是没有必要的

9 附录

9.1 实验运行代码

```
1 import requests
2 from tqdm import *
3 ciphertext = '''46307250616464316e674f7261636c33
4 9ae0735429869542efc40dcdc3f4c170
5 649463f719c5ddf4ce8c6d1ef0e5d41a
6 c5d137629e3fe1340cfaad7e21d65d14'''
7 cipher = [ciphertext[:32], ciphertext[32:64], ciphertext[64:96], ciphertext
8           [96:128]]
9 #获取第i到j字节
10 def get_bytes(s,i,j):
11     if j < i:
12         return ""
13     else:
14         return s[2*i:2*i+2*(j-i+1)]
15 #str转成列表
16 def get_list(s):
17     lst = [s[i:i+2] for i in range(0, len(s), 2)]
18     return lst
19 #列表转成str
20 def list_to_str(lst):
21     return ''.join(f'{x & 0xff:02x}' for x in lst)
22 p = []
23 for idx in trange(2,-1,-1):
24     y = cipher[idx+1]
25     r = cipher[idx]
26     #恢复只涉及最后两块，前面都设成0就行
27     pfx = 64*'0'
28     rb = int(get_bytes(r,15,15),16)
29     #最后一块不用遍历rb (r15)
30     if(idx != 2):
31         for i in trange(256):
32             c = pfx + r[:-2] + f'{i:02x}' + y
33             url = f"http://10.102.33.67:8208/dec_2?data={c}"
34             response = requests.get(url)
35             if response.status_code == 200:
36                 rb = i
37                 break
38     #修改r，前面的块这时候至少填充为1了
39     r = r[:-2] + f'{rb:02x}'
40 padding_length = 1
41 for j in trange(14,-1,-1):
42     #修改rj，比如都加1
43     rj = (int(get_bytes(r,j,j),16) + 1) & 0xff
44     c = pfx + get_bytes(r,0,j-1) + f'{rj:02x}' + get_bytes(r,j+1,15) + y
45     url = f"http://10.102.33.67:8208/dec_2?data={c}"
```

```
45     response = requests.get(url)
46     if response.status_code == 500:
47         padding_length += 1
48     else:
49         print("填充长度是",padding_length)
50         break
51 #填充长度是k, 那么把a_{16-k}一直到a15还原出来
52 a = get_list(get_bytes(r,16-padding_length,15))
53 a = [int(a[i],16) ^ padding_length for i in range(len(a))]
54 for j in range(16-padding_length):
55     #修改r, 使得填充变成padding_length+1
56     rr = [a[i] ^ (padding_length+1) for i in range((len(a)))]
57     #搜索r_{j-1},恢复a_{j-1}
58     rj_1 = 0
59     for i in trange(256):
60         r_ = get_bytes(r,0,14-padding_length)+f'{i:02x}'+list_to_str(rr)
61         c = pfx + r_ + y
62         url = f"http://10.102.33.67:8208/dec_2?data={c}"
63         response = requests.get(url)
64         if response.status_code == 200:
65             rj_1 = i
66             break
67     a = [rj_1 ^ (padding_length+1)] + a
68     padding_length +=1
69 r_n = get_list(r)
70 p = [int(r_n[i],16) ^ a[i] for i in range(len(a))] + p
71 for i in range(len(p)):
72     print(chr(p[i]),end='')
```